Índice

1. Ejercicio 1. Gestión de Plantas (DDL y DML)
2. Ejercicio 2. Gestión de Rutinas de Gimnasio (DAO Básico)
3. Ejercicio 3. Patrón DAO (Data Access Object)

Ejercicios Unidad 24 - Bases de Datos Relacionales

> **Antes de empezar**
>
> Para realizar estos ejercicios, deberás descargar los recursos del enlace de proyecto_poo. Como puedes ver, la solución está compuesta de varios proyectos. Cada uno de ellos corresponde con un ejercicio, deberás implementar todo el código, tanto de la Main como de los métodos que se piden en cada ejercicio. Cada proyecto contiene el test correspondiente, que deberás pasar para comprobar que has hecho el ejercicio correctamente.

Ejercicio 1. Gestión de Plantas (DDL y DML)

Vamos a crear una pequeña gestión de base de datos para un vivero. En este primer ejercicio nos centraremos en crear la estructura de la base de datos y poblarla con datos iniciales utilizando **ADO.NET** y **SQLite**.

```
Ejercicio 1: Gestión de Plantas

Insertando datos de ejemplo...

Insertando -> Rosa, Rosa spp., 10.50, 50
Insertando -> Lirio, Lilium, 7.80, 80
Insertando -> Tulipán, Tulipa, 5.25, 100
Insertando -> Orquídea, Orchidaceae, 25.00, 20
Datos insertados correctamente.
```

Requisitos

1. **Crea** un nuevo proyecto de consola y añade el paquete NuGet `Microsoft.Data.Sqlite`.
2. **Crea** una conexión a una base de datos llamada `vivero.db` en el `Main`.
3. **Implementa un método** `CreaTabla(SqliteConnection conexion)` que reciba la conexión abierta y cree la tabla `planta` (si no existe) con las columnas:
   - `id`: Entero, Clave Primaria y Autoincremental.
   - `nombre_comun`: Texto, no nulo.
   - `nombre_cientifico`: Texto, puede ser nulo o vacío.
   - `precio`: Real.
   - `stock`: Entero.
4. **Implementa un método** para insertar datos (ej. `InsertaPlanta(...)`) que reciba la conexión y los datos necesarios.
5. **Inserta** al menos **3 plantas** diferentes llamando a este método.
   - **Importante**: Debes utilizar **consultas parametrizadas** (usando `@parametro`) para evitar la inyección SQL.

Pistas

- Usa `using SqliteConnection ...` para gestionar la conexión correctamente.
- Usa sentencias SQL `CREATE TABLE IF NOT EXISTS...` y `INSERT INTO...`.
- Recuerda usar `comando.ExecuteNonQuery()` para las operaciones que no devuelven datos.

> **"Explora tu Base de Datos"**
>
> Una vez ejecutado el programa, se creará el archivo `vivero.db`. ¡No lo abras con un editor de texto!
> Te recomendamos utilizar una extensión de VS Code como **SQLite Viewer** (tal y como se indica en los apuntes) para abrir el archivo `.db` y comprobar visualmente que la tabla y los registros se han creado correctamente.

Ejercicio 2. Gestión de Rutinas de Gimnasio (DAO Básico)

Vamos a crear una aplicación para gestionar una rutina de ejercicios de gimnasio. En este ejercicio, nos centraremos en aplicar el patrón **DAO** para separar la lógica de acceso a datos de la lógica de negocio, usando enumeraciones y DTOs.

```
Ejercicio 2: Gestión de Rutinas de Gimnasio

Creando e insertando ejercicios...
Ejercicios que trabajan el músculo 'Pecho':
- Press de Banca
- Flexiones

Actualizando 'Flexiones' a 'Flexiones Diamante'...
Eliminando 'Press de Banca'...

Listado final de ejercicios:
- Flexiones Diamante (Músculos: Pecho, Triceps)
- Sentadillas (Músculos: Cuadriceps, Gluteos)
- Dominadas (Músculos: Espalda, Biceps)

Operaciones completadas.
```

Requisitos

1. **Enumeración de Músculos**:
   - Define una enumeración pública `Musculo` con al menos 5 valores (ej. `Pecho`, `Espalda`, `Biceps`, `Triceps`, `Cuadriceps`, `Gluteos`).
2. **Modelo de Datos**:

- Crea un record `Ejercicio` con las propiedades: `Id` (int), `Nombre` (string) y `GruposMusculares` (una `List<Musculo>`).

3. **Creación de Tablas**:
   - Implementa un método estático y público `CreaTabla(SqliteConnection conexion)` que cree la tabla `ejercicios` si no existe. La columna `grupos_musculares` deberá ser de tipo `TEXT` para almacenar la lista de músculos (p. ej., como una cadena separada por puntos y comas).
4. Crea una clase de utilidad EjeciciosConverter donde incluiremos todos los métodos de extensión que nos permitan cambiar de un tipo enumerado Musculo a cadena y a la inversa, los métodos se llamarán `ToMusculo` y `ToCadena`
5. **Clase DAO (`EjercicioDAO`)**:
   - Crea la clase `EjercicioDAO` que implemente `IDisposable`.
   - El constructor recibirá la cadena de conexión y gestionará el objeto `SqliteConnection`.
   - Implementa los siguientes métodos **CRUD**:
     - `Crea(Ejercicio ejercicio)` : Inserta un nuevo registro. Deberás convertir la `List<Musculo>` a una cadena para guardarla.
     - `Lee()` : Devuelve un `IEnumerable<Ejercicio>`. Deberás convertir la cadena de la base de datos de nuevo a una `List<Musculo>`.
     - `Actualiza(Ejercicio ejercicio)` : Modifica un ejercicio existente.
     - `Elimina(int id)` : Borra un ejercicio por su `Id`.
6. **Consulta con DTO**:
   - Crea un record `EjerciciosPorMusculo` con las propiedades `Musculo` (de tipo `Musculo`) y `NombresEjercicios` (una `List<string>`).
   - Implementa en el DAO un método `ObtenEjerciciosPorMusculo(Musculo musculo)` que devuelva un objeto `EjerciciosPorMusculo`. Este método buscará en la base de datos todos los ejercicios que contengan el músculo especificado en su lista de grupos musculares.
7. **Programa Principal (`Main`)**:
   - Define la cadena de conexión a `gimnasio.db` y llama al método para crear las tablas.
   - Crea una instancia de `EjercicioDAO` dentro de un bloque `using`.
   - Inserta al menos 3 ejercicios, cada uno con uno o más músculos asociados.

```
Ejercicio(0, "Press de Banca", new List<Musculo> { Musculo.Pecho });
Ejercicio(0, "Flexiones", new List<Musculo> { Musculo.Pecho, Musculo.Triceps });
Ejercicio(0, "Sentadillas", new List<Musculo> { Musculo.Cuadriceps, Musculo.Gluteos });
Ejercicio(0, "Dominadas", new List<Musculo> { Musculo.Espalda, Musculo.Biceps });
```

   - Llama a `ObtenEjerciciosPorMusculo` para obtener y mostrar todos los ejercicios que trabajan un músculo concreto (p. ej., `Musculo.Pecho`).

- Llama a `Actualiza` para modificar uno de los ejercicios.
- Llama a `Elimina` para borrar otro.
- Finalmente, llama a `Lee()` para obtener y mostrar la lista completa de ejercicios restantes, incluyendo sus grupos musculares.

Pistas:

- Para guardar la lista de enums, puedes usar `String.Join(",", ejercicio.GruposMusculares)`.
- Para leerla, puedes usar `cadena.Split(',').Select(e => Enum.Parse<Musculo>(e)).ToList()`.
- Para la consulta del DTO, puedes usar `WHERE` con el operador `LIKE` de SQL (ej. `WHERE grupos_musculares LIKE '%Pecho%'`).

Ejercicio 3. Patrón DAO (Data Access Object)

Vamos a refactorizar el ejercicio anterior para seguir estrictamente el patrón **DAO** (Data Access Object) tal como se explica en los apuntes de la unidad. Esto nos permitirá separar completamente la lógica de acceso a datos de la lógica de negocio. ***Recuerda crear cada entidad en ficheros distintos, así como los DAO.***

```
Ejercicio 3: Patrón DAO (Data Access Object)

Vivero: Vivero El Jardín
Dirección: Calle de las Flores 123
Inventario Valorado:
Petunia: 150 unidades - Valor Total: 375,00 €
Geranio: 80 unidades - Valor Total: 240,00 €

Operaciones completadas.
```

Requisitos

1. **Clases Entidad**: Separa los datos de la lógica.
   - `Planta`: Propiedades (`Id`, `NombreComun`, `NombreCientifico`, `Precio`, `Stock`).
   - `Vivero`: Propiedades (`Nombre`, `Direccion`) y una lista de plantas (`List<Planta>`).
2. **Clase DAO**: Crea una clase `PlantaDAO` que implemente `IDisposable` para gestionar la conexión automáticamente.
   - Debe recibir la cadena de conexión en el constructor y abrir la conexión.

- Debe implementar `Dispose()` para cerrar la conexión.

3. **Operaciones CRUD**: Implementa los siguientes métodos en `PlantaDAO`:
   - `Create(Planta p)`: Inserta una nueva planta.
   - `Read()`: Devuelve `IEnumerable<Planta>` con todas las plantas (usa `yield return`).
   - `Read(int id)`: Devuelve `Planta?` buscando por ID.
   - `Update(Planta p)`: Actualiza una planta existente. Comprueba antes si existe con `Read(id)`.
   - `Delete(int id)`: Borra una planta por ID. Comprueba antes si existe.
   - `Count()`: Devuelve el número total de plantas (usa `ExecuteScalar`).
4. **Consulta con DTO (Valor del Stock)**:
   - Crea un record `PlantaStock` con las propiedades: `Nombre_Comun`, `Stock` y `ValorTotal` (este último será el cálculo de precio * stock).
   - Implementa en el DAO un método `ObtenerInventario()` que devuelva `IEnumerable<PlantaStock>`.
   - La consulta SQL debe proyectar los campos necesarios y calcular el valor total (`SELECT ..., precio * stock as valor ...`).
5. **Programa Principal**:
   - **Carga JSON**: Lee el contenido del fichero `vivero.json` y deserialízalo en un objeto de la clase `Vivero` (que contendrá el nombre, dirección y las plantas iniciales).
   - **Instancia el DAO**: Utiliza `using PlantaDAO dao = new(...)`.
   - **Carga BD**: Si la base de datos está vacía (`dao.Count() == 0`), recorre las plantas del objeto `Vivero` y guárdalas una a una usando `dao.Create()`.
   - **Manipulación**: Realiza pruebas de modificación (`Update` / `Delete`) utilizando el DAO.
   - **Informe Final**: Genera un informe por consola que muestre cabecera con el **Nombre y Dirección del Vivero** (del objeto Vivero) y luego el listado de valoración del inventario (obtenido del método `ObtenerInventario` del DAO).

> ⚠️ **Mantén el método de crear la tabla fuera del DAO o como un método utilitario separado, ya que el DDL no suele formar parte del DAO estándar.**